

Introduction

Computer Graphics is a field of computer science that focuses on generating and manipulating visual content using programming. It is widely used in applications such as games, simulations, and graphical user interfaces.

The aim of this project is to design and implement a graphical scene using C++ and the graphics.h library. The scene represents a space environment that includes multiple objects such as a spaceship, planets, stars, and a moon.

The project is implemented in two versions:

A Static Scene, where all objects are drawn once and remain unchanged.

A Dynamic Scene, where objects move over time to create animation.

This project helps in understanding the difference between static and dynamic graphics and demonstrates how animation can be implemented using programming.

Methodology

The project was developed using a modular programming approach, where each graphical object is implemented as a separate function. This improves code organization, readability, and maintainability.

- **Tools and Technologies :**

Programming Language: C++

Graphics Library: graphics.h

Mathematical Functions: `sin()` and `cos()`

- **System Design and Implementation :**

The program is structured into several main functions, each responsible for drawing a specific object:

1. Spaceship Function (`drawSpaceship`)

This function draws the spaceship using basic graphical primitives:

A rectangle for the main body

Lines for the head and wings

A circle for the window

Flood fill to apply colors

Additional shapes to simulate the engine flame

2. Stars Function (drawStars)

Small stars are drawn using filled ellipses

Stars are distributed on both sides of the screen

They remain static in both versions

3. Planets Function (drawPlanets)

Two planets are drawn using filled shapes with different colors

Additional details like rings and lines are added

In the dynamic version, planets move using trigonometric functions:

$\cos(\text{angle})$ for horizontal movement

$\sin(\text{angle})$ for vertical movement

This creates circular motion

4. Moon Function (drawMoon)

The moon is drawn using a large filled shape

Small circles are added to represent craters

Positioned at the top-right corner

Algorithm

- **Static Scene :**

Initialize the graphics window

Draw stars

Draw planets

Draw the moon

Draw the spaceship at a fixed position

Wait for user input

- **Dynamic Scene :**

Initialize the graphics window

Set initial angle value
Start animation loop
Clear the screen
Draw stars
Draw the moon
Draw planets using updated angle
Draw spaceship moving upward
Update angle value
Add delay for smooth animation
Repeat until the loop ends

Discussion

This project demonstrates the fundamental concepts of computer graphics through the implementation of both static and dynamic scenes. It highlights how the addition of time and motion can significantly enhance the visual experience.

In the static version, the scene is displayed once without any changes, making it simple and efficient. In contrast, the dynamic version introduces animation by continuously updating and redrawing the scene, which creates a more engaging and realistic visual effect.

The animation in this project is based on the concept of frame redrawing, where the screen is cleared and redrawn repeatedly with updated object positions. In addition, trigonometric functions such as sine and cosine are used to create smooth circular motion for the planets, demonstrating the integration of mathematical concepts in computer graphics.

Difference between static and dynamic versions :

- **In the static version :**

All functions are called once to draw the scene without any movement.

- **In the dynamic version :**

A loop is used to create animation, where:

The screen is cleared using `cleardevice()`

Objects are redrawn in each frame

The spaceship moves upward

The planets move in circular paths

A delay is used to control animation speed

- **Challenges Faced :**

Overlapping of stars in some areas due to manual coordinate placement

Difficulty in positioning objects without overlap (e.g., spaceship and planets)

Maintaining visual balance in the scene

Controlling animation speed to ensure smooth motion

Avoiding flickering during frame updates

These challenges were addressed by adjusting coordinates, refining object placement, and carefully controlling timing to achieve smoother animation.

Overall, the project demonstrates how basic graphics techniques and mathematical functions can be combined to create a complete animated scene.

Source Code

- **Static :**

```
#include <iostream>
#include "graphics.h"
#include <math.h>
using namespace std;
// دالة رسم المركبة
void drawSpaceship(int x, int y)
{
    setcolor(WHITE);
    // جسم المركبة
    rectangle(x - 25, y - 60, x + 25, y + 40);
    setfillstyle(SOLID_FILL, BLUE);
    floodfill(x, y, WHITE);
    // رأس المركبة
    line(x - 25, y - 60, x, y - 110);
    line(x + 25, y - 60, x, y - 110);
    setfillstyle(SOLID_FILL, RED);
    floodfill(x, y - 70, WHITE);
    // الأجنحة
    line(x - 25, y, x - 50, y + 40);
    line(x - 50, y + 40, x - 25, y + 40);
    line(x + 25, y, x + 50, y + 40);
    line(x + 50, y + 40, x + 25, y + 40);
}
```

```

setfillstyle(SOLID_FILL, RED);
floodfill(x - 30, y + 35, WHITE);
floodfill(x + 30, y + 35, WHITE);
// النافذة
circle(x, y - 20, 12);
setfillstyle(SOLID_FILL, CYAN);
floodfill(x, y - 20, WHITE);
// النار المحركة
setcolor(WHITE);
line(x - 15, y + 40, x, y + 90);
line(x + 15, y + 40, x, y + 90);
setfillstyle(SOLID_FILL, YELLOW);
floodfill(x, y + 60, WHITE);
}
// دالة رسم النجوم
void drawStars()
{
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL, YELLOW);
    // النجوم على اليمين
    fillellipse(670, 170, 3, 3);
    fillellipse(580, 270, 4, 4);
    fillellipse(700, 360, 3, 3);
    fillellipse(610, 460, 4, 4);
    fillellipse(700, 560, 3, 3);
    // النجوم على اليسار
    fillellipse(70, 160, 3, 3);
    fillellipse(200, 240, 4, 4);
    fillellipse(90, 340, 3, 3);
    fillellipse(190, 440, 4, 4);
    fillellipse(100, 550, 3, 3);
}
// دالة رسم الكواكب (ثابتة)
void drawPlanets()
{
    // --- [ الكوكب الأول: البنفسي - أعلى اليسار ] ---
    int p1_x = 150;
    int p1_y = 120;
    // رسم الكوكب
    setcolor(COLOR(155, 89, 182));
    setfillstyle(SOLID_FILL, COLOR(155, 89, 182));
    fillellipse(p1_x, p1_y, 40, 40);
    // رسم الحلقة حول الكوكب (لون رمادي فاتح)
    setcolor(LIGHTGRAY);
    ellipse(p1_x, p1_y, 0, 360, 60, 15);
    // --- [ الكوكب الثاني: البرتقالي - أسفل اليمين ] ---
    int p2_x = 550;
    int p2_y = 400;
    // رسم الكوكب
    setcolor(COLOR(230, 126, 34));
    setfillstyle(SOLID_FILL, COLOR(230, 126, 34));
    fillellipse(p2_x, p2_y, 45, 45);
    // رسم الخطوط التفصيلية (لون برتقالي غامق)
    setcolor(COLOR(211, 84, 0));
    line(p2_x - 40, p2_y - 10, p2_x + 40, p2_y - 10);

```

```

    line(p2_x - 35, p2_y + 10, p2_x + 35, p2_y + 10);
}
// دالة رسم القمر (ثابت)
void drawMoon()
{
    int moon_x = 650;
    int moon_y = 100;
    // رسم جسم القمر الأساسي (رمادي فاتح)
    setcolor(COLOR(220, 220, 220));
    setfillstyle(SOLID_FILL, COLOR(220, 220, 220));
    fillellipse(moon_x, moon_y, 40, 40);
    // رسم الفوهات والتفاصيل (رمادي غامق)
    setcolor(COLOR(180, 180, 180));
    setfillstyle(SOLID_FILL, COLOR(180, 180, 180));
    // الفوهة الأولى
    fillellipse(moon_x - 10, moon_y - 15, 5, 5);
    // الفوهة الثانية
    fillellipse(moon_x + 16, moon_y + 11, 6, 6);
    // الفوهة الثالثة
    fillellipse(moon_x - 14, moon_y + 16, 6, 6);
}
int main()
{
    // فتح نافذة الرسم
    initwindow(800, 600, "Space Mission - Static Scene");
    // رسم جميع العناصر بالترتيب من الخلفية للأمام
    drawStars(); // رسم النجوم
    drawPlanets(); // رسم الكواكب
    drawMoon(); // رسم القمر في أعلى اليمين
    drawSpaceship(400, 300); // رسم المركبة في المنتصف
    // إبقاء الشاشة مفتوحة
    getch();
    closegraph();
    return 0;
}

```

• Dynamic :

```

#include <iostream>
#include "graphics.h"
#include <math.h>
using namespace std;
// دالة رسم المركبة
void drawSpaceship(int x, int y)
{
    setcolor(WHITE);
    // جسم المركبة
    rectangle(x - 25, y - 60, x + 25, y + 40);
    setfillstyle(SOLID_FILL, BLUE);
    floodfill(x, y, WHITE);
    // رأس المركبة
    line(x - 25, y - 60, x, y - 110);
    line(x + 25, y - 60, x, y - 110);
}

```

```

setfillstyle(SOLID_FILL, RED);
floodfill(x, y - 70, WHITE);
// الأجنحة
line(x - 25, y, x - 50, y + 40);
line(x - 50, y + 40, x - 25, y + 40);
line(x + 25, y, x + 50, y + 40);
line(x + 50, y + 40, x + 25, y + 40);
setfillstyle(SOLID_FILL, RED);
floodfill(x - 30, y + 35, WHITE);
floodfill(x + 30, y + 35, WHITE);
// النافذة
circle(x, y - 20, 12);
setfillstyle(SOLID_FILL, CYAN);
floodfill(x, y - 20, WHITE);
// النار المحركة
setcolor(WHITE);
line(x - 15, y + 40, x, y + 90);
line(x + 15, y + 40, x, y + 90);
setfillstyle(SOLID_FILL, YELLOW);
floodfill(x, y + 60, WHITE);
}
// دالة رسم النجوم (ثابتة في مكانها)
void drawStars()
{
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL, YELLOW);
    // النجوم على اليمين
    fillellipse(670, 170, 3, 3);
    fillellipse(580, 270, 4, 4);
    fillellipse(700, 360, 3, 3);
    fillellipse(610, 460, 4, 4);
    fillellipse(700, 560, 3, 3);
    // النجوم على اليسار
    fillellipse(70, 160, 3, 3);
    fillellipse(200, 240, 4, 4);
    fillellipse(90, 340, 3, 3);
    fillellipse(190, 440, 4, 4);
    fillellipse(100, 550, 3, 3);
}
// دالة رسم الكواكب (تحتوي على حركة دائرية)
void drawPlanets(float angle)
{
    // --- [ الكوكب الأول: البنفسي - أعلى اليسار ] ---
    int p1_x = 150 + (int)(cos(angle) * 20);
    int p1_y = 120 + (int)(sin(angle) * 20);
    setcolor(COLOR(155, 89, 182));
    setfillstyle(SOLID_FILL, COLOR(155, 89, 182));
    fillellipse(p1_x, p1_y, 40, 40);
    setcolor(LIGHTGRAY);

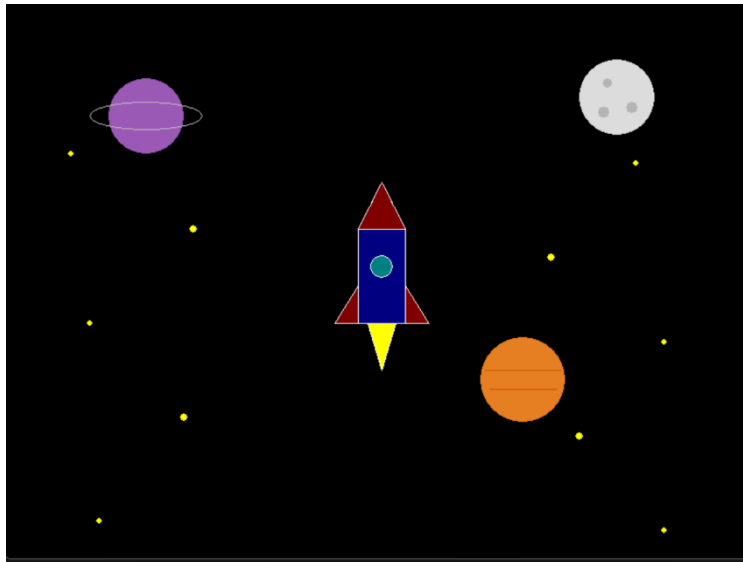
```

```

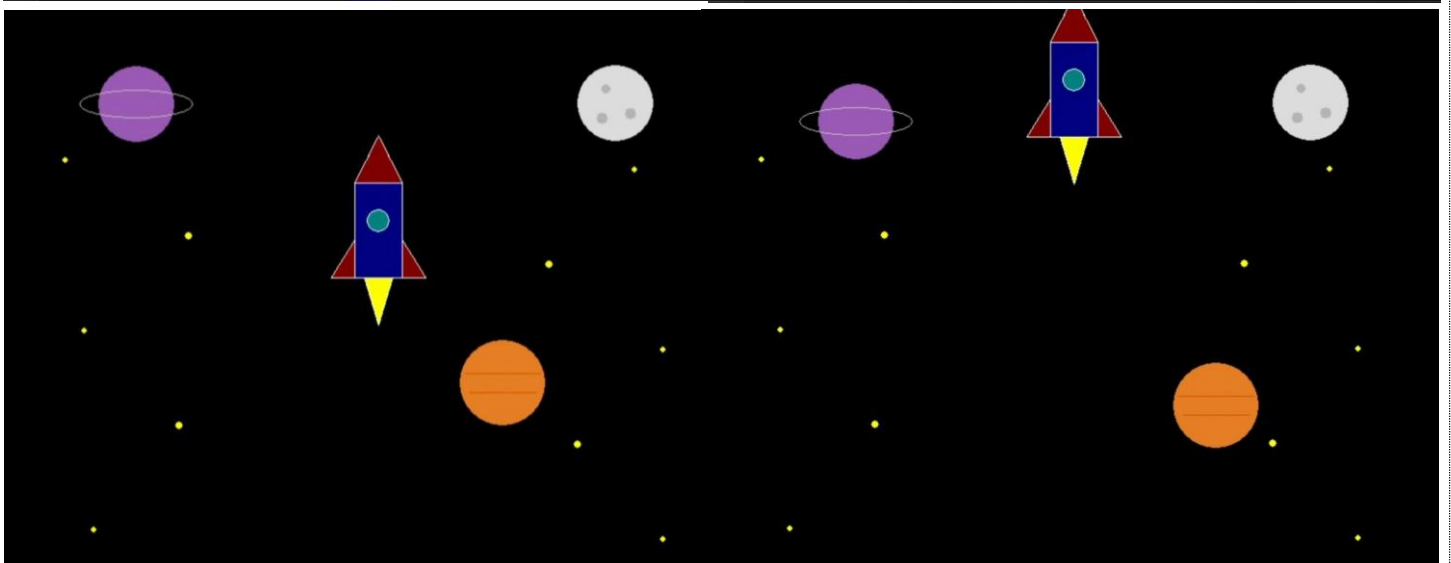
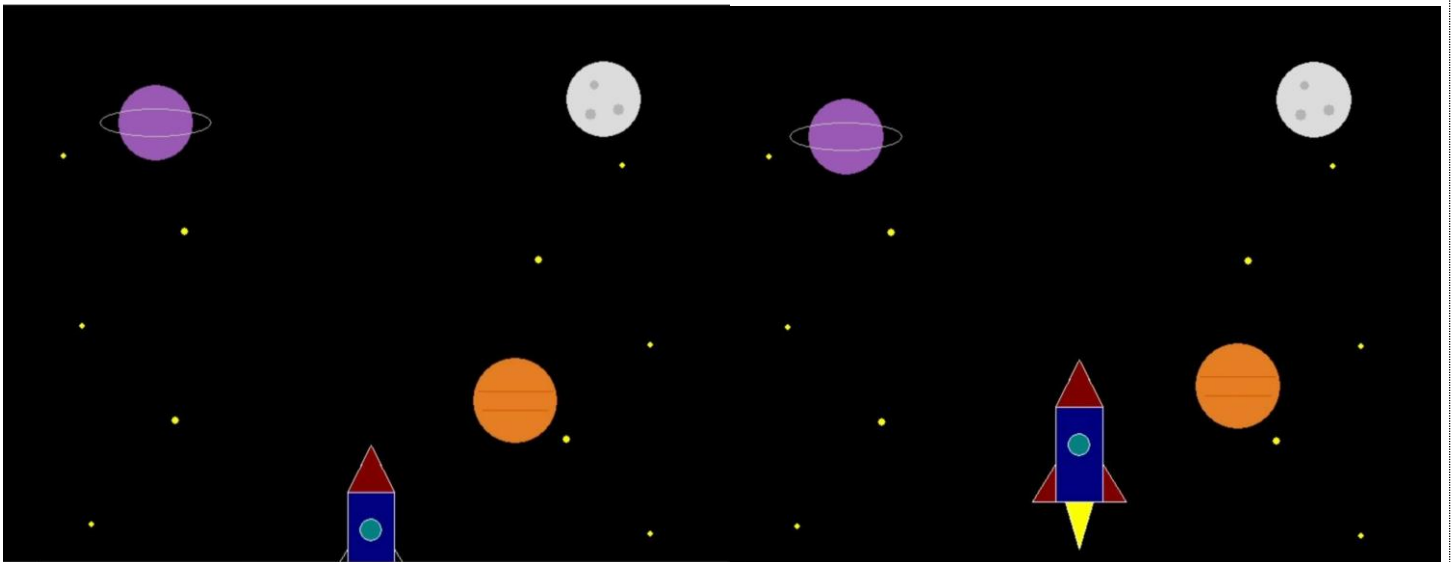
ellipse(p1_x, p1_y, 0, 360, 60, 15);
// --- [ الكوكب الثاني: البرتقالي - أسفل اليمين ] ---
int p2_x = 550 + (int)(sin(angle) * 20);
int p2_y = 400 + (int)(cos(angle) * 20);
setcolor(COLOR(230, 126, 34));
setfillstyle(SOLID_FILL, COLOR(230, 126, 34));
fillellipse(p2_x, p2_y, 45, 45);
setcolor(COLOR(211, 84, 0));
line(p2_x - 40, p2_y - 10, p2_x + 40, p2_y - 10);
line(p2_x - 35, p2_y + 10, p2_x + 35, p2_y + 10);
}
// دالة رسم القمر (ثابت)
void drawMoon()
{
    int moon_x = 650;
    int moon_y = 100;
    // رسم جسم القمر الأساسي
    setcolor(COLOR(220, 220, 220));
    setfillstyle(SOLID_FILL, COLOR(220, 220, 220));
    fillellipse(moon_x, moon_y, 40, 40);
    // رسم الفوهات
    setcolor(COLOR(180, 180, 180));
    setfillstyle(SOLID_FILL, COLOR(180, 180, 180));
    fillellipse(moon_x - 10, moon_y - 15, 5, 5);
    fillellipse(moon_x + 16, moon_y + 11, 6, 6);
    fillellipse(moon_x - 14, moon_y + 16, 6, 6);
}
int main()
{
    // فتح نافذة الرسم
    initwindow(800, 600, "Space Mission - Dynamic Scene");
    float angle = 0.0f; // متغير زاوية دوران الكواكب
    int shipX = 400; // موقع المركبة الأفقي ثابت في النص
    // حلقة التحريك (Animation Loop)
    for (int shipY = 600; shipY > -150; shipY -= 4)
    {
        cleardevice(); // مسح الشاشة لإعادة الرسم في كل إطار
        drawStars(); // رسم النجوم (خلفية ثابتة). 1.
        drawMoon(); // رسم القمر (ثابت). 2.
        drawPlanets(angle); // رسم الكواكب (متحركة). 3.
        drawSpaceship(shipX, shipY); // 4. رسم المركبة (متحركة للأعلى).
        // تحديث زاوية الكواكب
        angle += 0.05f;
        if (angle > 2 * 3.14159f) angle = 0.0f;
        delay(20); // ضبط سرعة الحركة
    }
    getch();
    closegraph();
    return 0; }

```


Screenshots



Static Scene



Dynamic Scene

Conclusion

In this project, a space scene was successfully designed and implemented in both static and dynamic forms using C++ and the graphics.h library.

The project demonstrates key concepts such as drawing shapes, managing coordinates, and creating animation using loops and mathematical functions. The use of sine and cosine functions enabled smooth circular motion, enhancing the realism of the scene.

This project provided valuable practical experience in computer graphics and helped build a strong foundation for developing more advanced graphical applications.